COMPTE RENDU DE TP

TP3 - Tests automatisés

ThermoNode | Firmware embarqué et passerelle MQTT

Projet	ThermoNode	**Dépôt**	thermonode-binome
Environnement	Ubuntu 24.04.3 dans Vagrant, hôte Windows	**Langages**	C et Python
Outils	Unity, pytest, gcovr, Cppcheck, Docker Compose	**Date**	22 octobre 2026

Résumé des résultats

76	22	93,6 %	13 / 20
tests firmware passés	tests passerelle passés	lignes firmware couvertes	exigences tracées

Ce rapport retrace les étapes du TP à partir des commandes, sorties et captures d’écran partagées. Huit défauts ont été révélés puis corrigés dans des branches Git dédiées. Les tests unitaires firmware et passerelle passent. La matrice laisse sept exigences sans test associé; le test automatisé de reconnexion MQTT n’est pas confirmé comme réussi.

Remarque : les captures ci-dessous sont les images de terminal transmises pendant le travail. Elles documentent les étapes de découverte, les résultats intermédiaires et la traçabilité.

1. Mise en place et prise en main

Le projet a été préparé dans la VM Ubuntu via Vagrant. Un dépôt Git local a été initialisé, puis relié au dépôt GitHub. Les premières commandes confirment la disponibilité des outils et l'existence du squelette firmware et passerelle.

Commandes de démarrage et versionnement

```
# Sur Windows, depuis le dossier du projet
vagrant up
vagrant ssh

# Dans Ubuntu, à la racine ~/tn
git config --global user.name "Mohamedamine Ayouni"
git config --global user.email "[adresse configurée localement]"
git init -q
git add -A
git commit -m "TP0: squelette ThermoNode"
git branch -M main
git remote add origin https://github.com/mohamedamineAyouni/thermonode-binome.git
git push -u origin main
```

Les outils vérifiés comprenaient gcc, make, cppcheck, arm-none-eabi-gcc, Python, gcovr, pytest, Docker Compose et Mosquitto. Le dépôt a ensuite reçu des commits de tests et de corrections, avec une branche par défaut détecté.

Dépannage de la VM et du stockage Docker

Au retour d une séance, Vagrant a signalé un conflit sur le port 8080, puis une attente SSH expirée alors que la VM était marquée running. Après arrêt forcé et redémarrage, SSH a de nouveau fonctionné. Le conteneur gateway a ensuite échoué à ouvrir SQLite dans /data; le Dockerfile a été complété pour créer le dossier et le rendre accessible à l utilisateur gw.

```
# Depuis Windows PowerShell, dans le dossier du Vagrantfile
vagrant status
vagrant halt --force
vagrant up
vagrant ssh

# Contrôle de la VM depuis l hôte
Test-NetConnection 127.0.0.1 -Port 2222

# Correction du stockage SQLite du conteneur
# gateway/Dockerfile : RUN mkdir -p /data && chown gw:gw /data
docker compose -f ci/compose.test.yaml run --rm --no-deps --user root --entrypoint sh gateway -c 'chown -R gw:gw /data'
docker compose -f ci/compose.test.yaml up -d --build --wait
```

Contrôle initial du firmware et de la passerelle

```
make -C firmware build
make -C firmware test
make -C firmware coverage
make -C firmware static
make -C firmware size-check
cd ~/tn/gateway && pytest tests/unit -q
cd ~/tn && docker compose -f ci/compose.test.yaml up -d --build --wait
docker compose -f ci/compose.test.yaml ps
curl -s http://localhost:8000/health
mosquitto_sub -h localhost -t 'thermonode/#' -v
```

Le test firmware initial avait seulement deux tests; pytest unitaire avait une réussite. Le premier contrôle de couverture firmware donnait 2,5 % des lignes et 2,9 % des branches. Une exécution manuelle du simulateur a ensuite montré des messages MQTT NORMAL et ALERT.

Commandes de tests d exigences et journal

```
grep -n TODO firmware/test/hal_double.c
nano docs/journal-defauts.md
make -C firmware test 2>&1 | grep -E "FAIL|Tests"
make -C firmware clean
make -C firmware test SAN=1 2>&1 | grep -E "runtime error|AddressSanitizer|#0" | head -4
```

2. Détection des défauts - firmware

Les tests ont été ajoutés avant les corrections afin d'obtenir des symptômes observables. Le journal de défauts relie chaque identifiant à une exigence, un test révélateur, le message constaté, une hypothèse de cause et la technique utilisée.

DEF-01 à DEF-04

ID	Test / exigence	Symptôme et technique	Correction
DEF-01	test_EF05_range_check_bounds EF-05	La borne haute 125,00 °C était exclue. Valeurs limites.	Borne maximale rendue inclusive dans range_check().
DEF-02	test_EF03_transient_partial_window EF-03	Moyenne incorrecte et index 5 hors tableau. Séquences et sanitizer.	Boucle du filtre limitée aux éléments présents.
DEF-03	test_EF04_counter_resets_below_threshold EF-04	Compteur non remis à zéro sous seuil. Table de décision.	Réinitialisation du compteur dans le cas sous seuil.
DEF-04	test_EI02_crc_check_value_123456789_is_29B1 EI-02	CRC attendu 0x29B1, obtenu 0x31C3; CRC vide attendu 0xFFFF, obtenu 0x0000.	Initialisation CCITT-FALSE à 0xFFFF.

Commandes représentatives

```
# Déclencher et documenter les tests avant correction
make -C firmware test 2>&1 | grep -E "FAIL|Tests"
make -C firmware clean
make -C firmware test SAN=1 2>&1 | grep -E "runtime error|AddressSanitizer|#0" | head -12

# Branches et corrections dans l ordre du TP
git switch -c fix/DEF-01-range-upper-bound
sed -i 's/centi < TN_TEMP_MAX_CENTI/centi <= TN_TEMP_MAX_CENTI/' firmware/src/convert.c
# Correction filtre : i <= count devient i < count
# Correction threshold.c : remettre above à 0 quand l échantillon est sous seuil
sed -i 's/uint16_t crc = 0x0000u;/uint16_t crc = 0xFFFFu;/' firmware/src/frame.c
make -C firmware test
```

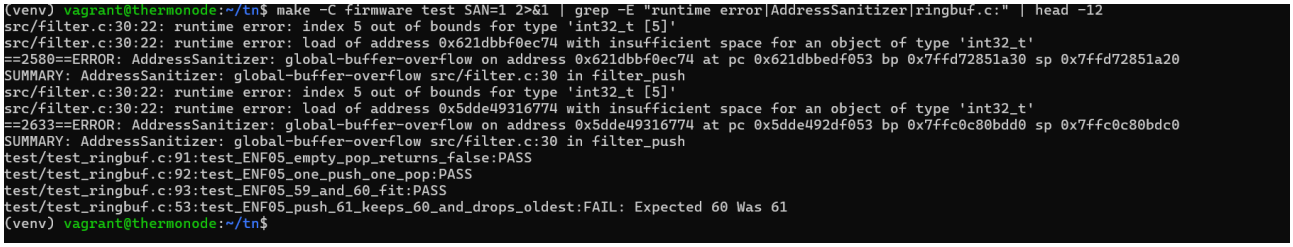


Figure 1. AddressSanitizer révèle l accès hors limites dans filter_push(); les tests du tampon révèlent aussi le défaut de capacité.

3. Détection des défauts - tampon, nœud et passerelle

Les tests de frontière du tampon, les transitions d état du nœud et les trames invalides de la passerelle ont complété le journal. Les tests initiaux font ressortir le comportement incorrect avant son activation ou sa correction.

ID	Test / exigence	Résultat avant correction	Correction
DEF-05	test_ENF05_push_61_keeps_60_and_drops_oldest ENF-05	61 éléments conservés au lieu de 60.	Condition pleine : count >= RINGBUF_CAP.
DEF-06	test_EF05_recovers_to_normal_after_5_valid_samples EF-05	Attendu NORMAL (1), obtenu FAULT (3).	Transition activée par fault_recover_ready().
DEF-07	test_EI02_invalid_frames_are_rejected EI-02	temp chaîne et seq négative acceptées.	Valider type numérique temp, refuser bool et seq négative.
DEF-08	test_ENF05_duplicate_seq_is_ignored ENF-05	Doublons et anciennes séquences acceptés.	Ignorer seq <= dernier numéro stocké.

Commandes et corrections représentatives

```
# Tampon circulaire
sed -i 's/count > RINGBUF_CAP/count >= RINGBUF_CAP/' firmware/src/ringbuf.c

# Reprise nœud
sed -i 's/if (false) {   \/\* TODO reprise \*\//if (fault_recover_ready()) {/' firmware/src/node.c

# Validation de trame gateway
sed -i 's/if isinstance(seq, bool) or not isinstance(seq, int):/if isinstance(seq, bool) or not isinstance(seq, int) or seq < 0
# Température : refuser les types autres que int/float, et bool

# Dédoublonnage
sed -i 's/if False and last is not None and f.seq <= last:/if last is not None and f.seq <= last:/' gateway/gateway/handler.py
cd ~/tn/gateway && pytest tests/unit -q
cd ~/tn && make -C firmware test
```

Une ligne de commande de capture avait été saisie avec une faute de frappe pendant les essais; le code source de production a été corrigé et contrôlé séparément. Le commit DEF-08 utilisé est c02db94. Les correctifs firmware ont été fusionnés dans main.

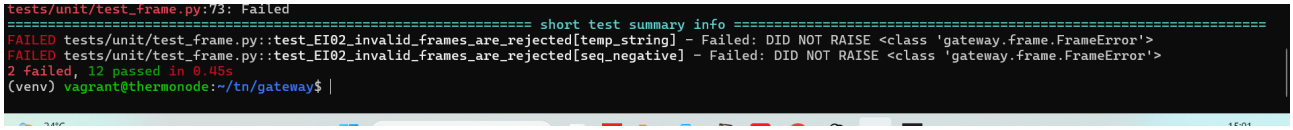


Figure 2. Tests gateway paramétrés : les cas temp_string et seq_negative révèlent les contrôles manquants.

4. Organisation Git et preuves intermédiaires

Chaque défaut a été travaillé sur une branche fix/DEF-xx, corrigé, soumis à nouveau aux tests, puis fusionné dans main. Les commits correspondants portent un message explicite test(...) ou fix(...). Les défauts ont été décrits dans docs/journal-defauts.md.

```
# Exemple de cycle suivi pour les corrections
git switch -c fix/DEF-xx-nom
# Modifier le fichier concerné et relancer les tests ciblés
git add <fichiers>
git commit -m "fix(...): ... (DEF-xx, exigence)"
git switch main
git merge --no-ff fix/DEF-xx-nom -m "merge: DEF-xx"

# Tests firmware ciblés et suite complète
make -C firmware test 2>&1 | grep -E "test_<module>|Tests|FAIL|error:"
make -C firmware clean && make -C firmware test
```

```
(venv) vagrant@thermonode:~/tn$ cd ~/tn/firmware
ASAN_OPTIONS=abort_on_error=1:halt_on_error=1 ./build/test_ringbuf
test/test_ringbuf.c:91:test_ENF05_empty_pop_returns_false:PASS
test/test_ringbuf.c:92:test_ENF05_one_push_one_pop:PASS
test/test_ringbuf.c:93:test_ENF05_59_and_60_fit:PASS
test/test_ringbuf.c:53:test_ENF05_push_61_keeps_60_and_drops_oldest:FAIL: Expected 60 Was 61
test/test_ringbuf.c:95:test_ENF05_fifo_order_on_drain:PASS
test/test_ringbuf.c:96:test_ENF05_peek_does_not_remove:PASS

-----------------------
6 Tests 1 Failures 0 Ignored
FAIL
(venv) vagrant@thermonode:~/tn/firmware$
```

Figure 3. Test de frontière ENF-05 : la 61e insertion conservait trop d éléments avant correction.

```
(venv) vagrant@thermonode:~/tn$ nano firmware/test/test_fault.c
(venv) vagrant@thermonode:~/tn$ make -C firmware test 2>&1 | grep -E "test_fault|Tests|error:"
11 Tests 1 Failures 0 Ignored
test/test_fault.c:66:test_no_fault_for_valid_sample:PASS
test/test_fault.c:67:test_fault_for_temperature_below_minimum:PASS
test/test_fault.c:68:test_fault_for_temperature_above_maximum:PASS
test/test_fault.c:69:test_not_frozen_before_five_identical_samples:PASS
test/test_fault.c:70:test_frozen_after_five_identical_samples:PASS
test/test_fault.c:71:test_different_adc_resets_frozen_count:PASS
test/test_fault.c:55:test_invalid_sample_resets_frozen_count:FAIL: Expected FALSE Was TRUE
test/test_fault.c:60:test_adc_out_of_range_is_fault:FAIL: Expected TRUE Was FALSE
8 Tests 2 Failures 0 Ignored
6 Tests 2 Failures 0 Ignored
11 Tests 2 Failures 0 Ignored
25 Tests 1 Failures 0 Ignored
6 Tests 1 Failures 0 Ignored
9 Tests 1 Failures 0 Ignored
(venv) vagrant@thermonode:~/tn$
```

Figure 4. Tests du détecteur de défaut : les cas MC/DC révèlent deux comportements à corriger.

Les sorties de test ont été ajoutées au journal avant d appliquer les corrections, puis les tests ciblés ont été relancés. Les tests firmware finaux sont présentés plus loin.

5. Contrôles finaux exécutés

Après fusion des corrections, les commandes suivantes ont fourni les résultats finaux partagés.

```
# Suite complète du firmware
make -C firmware clean
make -C firmware test

# Vérification mémoire
make -C firmware test SAN=1

# Couverture firmware
make -C firmware clean && make -C firmware coverage

# Tests et couverture passerelle
cd ~/tn/gateway
pytest tests/unit -q
pytest tests/unit -q --cov=gateway --cov-branch

# Matrice de traçabilité (depuis ~/tn)
cd ~/tn
python3 tools/traceability.py docs/requirements.md "reports/junit-*.xml" > reports/traceability.md
cat reports/traceability.md
```

Résultats observés

Mesure	Résultat final transmis
Unity firmware	76 tests, 0 échec : convert 11, fault 8, filter 6, frame 11, node 25, ringbuf 6, threshold 9.
Sanitizers	76 tests passés; aucun diagnostic AddressSanitizer / runtime error dans la sortie transmise.
Firmware coverage	93,6 % lignes (264/282); 89,9 % branches (124/138); 92,1 % fonctions (35/38).
Firmware size	Flash 10 148 / 49 152 octets (20,6 %); RAM 1 640 / 8 192 octets (20,0 %).
Pytest gateway	22 tests passés, 0 échec.
Gateway coverage	79 % global; frame 89 %, handler 98 %, health 0 %, storage 84 %, __init__ 100 %.

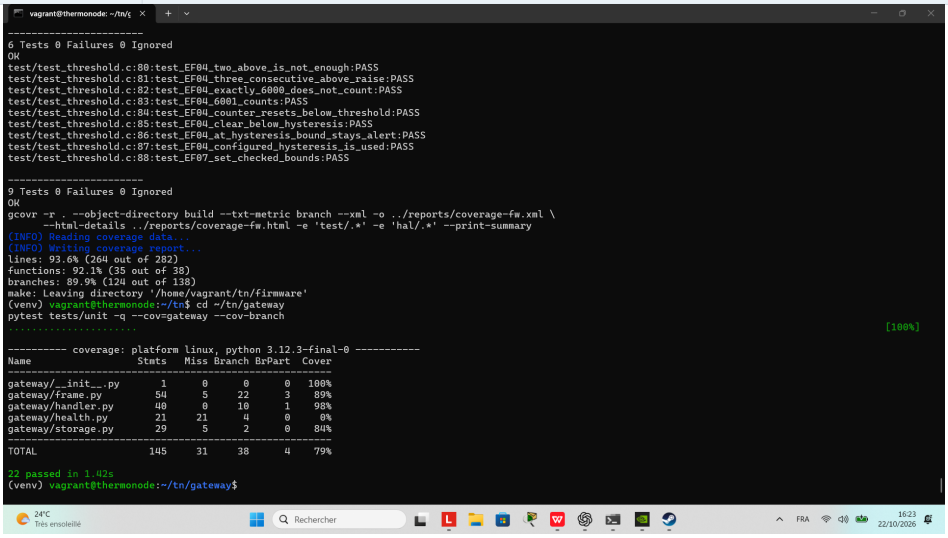


Figure 5. Capture de la couverture de la passerelle : 22 tests réussis et 79 % de couverture.

6. Matrice de traçabilité et exigences non couvertes

La matrice générée par tools/traceability.py compte 20 exigences. Treize sont associées à des tests passés, soit 65 %. Sept exigences restent orphelines dans les rapports JUnit. Ce statut signifie qu aucun test n a été reconnu pour elles dans cette génération de matrice; elles ne doivent pas être présentées comme validées.

Statut	Exigences
Couvertes - 13	EF-01 à EF-08; EI-01 à EI-03; ENF-01; ENF-05.
Orphelines - 7	C-01, C-02, C-03, ENF-02, ENF-03, ENF-04, ENF-06.

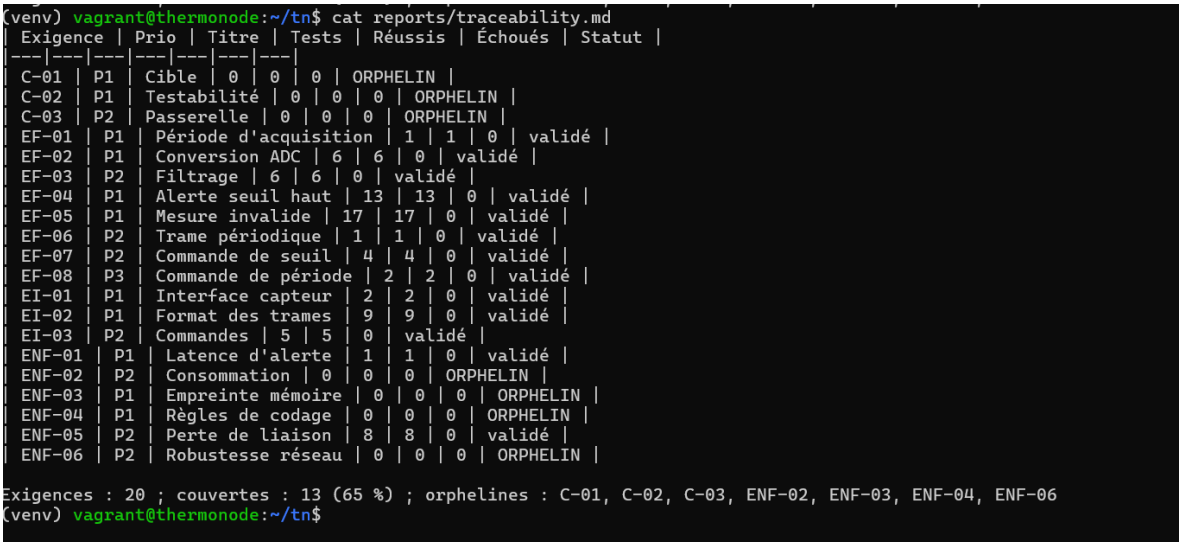


Figure 6. Matrice de traçabilité affichée dans le terminal après génération.

La capture détaillée ci-dessous montre également le bilan de 13 exigences couvertes sur 20.

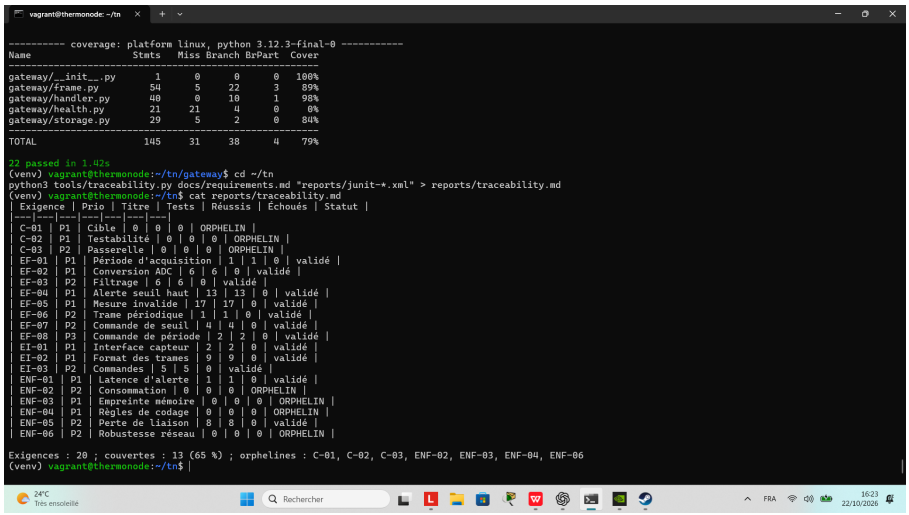


Figure 7. Sortie complète du rapport de traçabilité, avec les exigences orphelines nommées.

7. Intégration, limites et conclusion

Essais Docker et MQTT

Les services broker, gateway et node-sim ont été démarrés avec Docker Compose. L endpoint /health a répondu « ok » et le client Mosquitto a observé des trames NORMAL puis ALERT. Un problème initial de permission SQLite a été corrigé en créant /data et en donnant la propriété au compte gw dans l image gateway. Le build Docker a ensuite réussi.

```
docker compose -f ci/compose.test.yaml ps
curl -s http://localhost:8000/health
mosquitto_sub -h localhost -t 'thermonode/#' -v

docker compose -f ci/compose.test.yaml up -d --build --wait
cd ~/tn/gateway
MQTT_HOST=localhost pytest tests/integration -q --timeout=180
```

Limite : le test automatise de reconnexion après coupure broker a expiré pendant le setup ou l attente des messages lors des exécutions partagées. Après les corrections finales CRC et dédoublonnage, il n y a pas de résultat vert de ce test dans les éléments transmis. Il reste donc à le relancer avant de conclure que ENF-05 est validée en intégration.

Conclusion

Le travail a suivi une démarche de test par défaut : écrire des cas révélateurs, conserver les résultats initiaux, documenter les anomalies, corriger sur des branches dédiées puis rejouer les suites. Huit défauts ont été corrigés. Les suites firmware (76 tests) et passerelle (22 tests) passent, la couverture firmware est de 93,6 % lignes et 89,9 % branches, et les contraintes de ressources restent sous les plafonds du projet.

Avant remise, il reste recommandé de : (1) réussir ou documenter le test d intégration de reconnexion; (2) ajouter des preuves pour les sept exigences orphelines ou expliquer leur statut selon l énoncé; (3) vérifier que les commits récents sont poussés vers GitHub. La dernière sortie Git partagée indiquait main en avance de 27 commits sur origin/main.

Historique des corrections fusionnées

DEF-01 a4cbd89; DEF-02 6c7b8cb; DEF-03 739d8ca; DEF-04 eb2ab64; DEF-05 55cf76a; DEF-06 83cbe90; DEF-07 5d238ad; DEF-08 c02db94. Les commits de tests et de documentation sont également présents dans l historique Git.

Annexe - fichiers de preuve

docs/journal-defauts.md; docs/mcdc-EF05.md; reports/junit-fw.xml; reports/coverage-fw.xml; reports/coverage-fw.html; reports/size.txt; reports/traceability.md.